BÁO CÁO KỸ THUẬT

HỆ THỐNG GIÁM SÁT VÀ PHÂN TÍCH VIDEO ĐA CAMERA

Multi-Camera Video Processing and Analytics System

Báo cáo kỹ thuật

Date: 10/2026

Thông tin phần cứng: Intel Core i7-8650u CPU Only 1.90GHz + 16 GB RAM

Github: Link

Thông tin

Thuộc tính	Nội dung
Tên hệ thống	Multi-Camera Video Processing and Analytics System
Mục đích	Xây dựng hệ thống multi-camera giám sát giao thông
Phạm vi kiểm tra	Xử lí multi-camera input nhiều luồng, thực hiện detect và tracking (theo line và zone)
Phiên bản vision đang cấu hình	YOLOv8s, OpenVINO IR, imgsz=480, CPU, conf=0.7, IoU/NMS=0.45.
Ngày khảo sát	24/10/2026.
Trạng thái	Pipeline đa camera và đường RTSP đã được kiểm thử riêng với MediaMTX cấu hình đúng. Log chạy đồng thời bốn camera cho thấy CPU chưa đáp ứng realtime (khoảng 0.04 inference FPS/camera, latency 22.5–25.6 giây); độ chính xác đếm chưa được đo do chưa có ground truth. Cần xử lý hiệu năng, xác nhận RTSP publisher ổn định và hiệu chuẩn ROI trước khi dùng production.

Mục lục

Thông tin 2
Mục lục 3
1. Mô tả hệ thống 4
2. Mục tiêu và phạm vi hệ thống 4
    2.1 Mục tiêu chức năng 4
    2.2 Phạm vi khảo sát 4
3. Kiến trúc tổng thể 4
    3.1 Sơ đồ luồng 5
    3.2 Cách ly theo camera 5
4. Công nghệ và cấu hình hiện tại 5
    4.1 Cấu hình model/processing 5
    4.2 Export model 6
5. Luồng xử lý video 6
    5.1 Capture 6
    5.2 Pipeline và metrics 6
    5.3 Restream 6
6. Phát hiện, tracking và analytics 6
    6.1 Detector và tracking 6
    6.2 Line crossing 6
    6.3 Vùng hạn chế 6
    6.4 Cấu hình ROI tạm thời 6
7. Lưu trữ, API và giao diện giám sát 7
    7.1 SQLite và snapshot 7
    7.2 REST API 7
    7.3 Desktop UI 7
8. Đánh giá và kết quả kiểm thử 7
    8.1 Functional tests 7
    8.2 Inference smoke test 7
    8.3 Độ chính xác nghiệp vụ chưa được đo 8
9. Hạn chế, rủi ro và khuyến nghị 8
    9.1 Kế hoạch tối ưu theo giai đoạn 8
10. Kết luận 8

1. Mô tả hệ thống

Hệ thống nhận video từ nhiều camera hoặc file, chạy phát hiện đối tượng YOLO và tracker theo từng camera, tính sự kiện cắt vạch/xâm nhập vùng, lưu sự kiện và snapshot vào SQLite, sau đó cung cấp luồng video gắn nhãn qua RTSP, MJPEG, REST API và giao diện desktop Qt. Cấu hình mẫu hiện bật bốn nguồn video 1920×1080.

Kiến trúc hiện sử dụng một YOLOv8s OpenVINO inference worker/API dùng chung, còn ByteTrack và analytics state tách riêng theo camera. RTSP đã được smoke-test với MediaMTX có cấu hình path publisher. Độ chính xác detection/counting chưa được đo vì chưa có ground truth.

Khía cạnh	Kết quả / trạng thái
Model	YOLOv8s COCO pretrained; OpenVINO export có sẵn và được cấu hình dùng trực tiếp.
Nguồn thử	Bốn video mẫu C001–C004, 1920×1080. RTSP smoke test được chạy riêng với MediaMTX đã cấu hình path publisher.
Cold-start inference	Phép đo pipeline một camera trước đây ghi nhận khoảng 12.8 giây ở lần đầu; chịu overhead khởi tạo/warm-up, không đại diện steady-state.
Steady-state CPU inference	Đo ngắn một camera trước đây khoảng 1.28 giây/frame (~0.78 inference FPS). Không dùng số này để dự đoán throughput bốn camera.
Pipeline output	Log đồng thời bốn camera gần nhất: mỗi camera khoảng 0.04 inference FPS, latency trung bình theo cửa sổ khoảng 22.5–25.6 giây; output khoảng 3.2–7.9 FPS. Không realtime; kết quả phụ thuộc phần cứng và cửa sổ đo.
Kiểm thử tự động	32 tests passed; Python compile check, cấu hình MediaMTX và git diff --check cũng qua. Có một deprecation warning Starlette/httpx.
Độ chính xác đếm	Chưa có ground truth gắn nhãn để tính precision/recall, IDF1 hoặc sai số đếm; chưa kết luận được counting chính xác.

2. Mục tiêu và phạm vi hệ thống

2.1 Mục tiêu chức năng

- Thu nhận video từ RTSP, webcam hoặc file video theo từng camera.
- Phát hiện các lớp COCO được cấu hình: person, bicycle, car, motorcycle, bus và truck.
- Theo dõi đối tượng liên tục trong phạm vi từng camera.
- Phát hiện hướng cắt vạch hai chiều và lần đầu xâm nhập polygon vùng hạn chế.
- Lưu sự kiện, snapshot, tổng đếm theo camera và truy vấn qua API.
- Phát lại video đã annotate và cung cấp dashboard desktop để giám sát.

2.2 Phạm vi khảo sát

Hệ thống đã được kiểm thử trên các video mẫu và có RTSP publish/read smoke test riêng với MediaMTX cấu hình đúng. Chưa đủ cơ sở để tuyên bố hệ thống sẵn sàng production, cần tiến hành retest lại trên các môi trường khác nhau để đảm bảo hiệu suất trước khi chạy production

3. Kiến trúc tổng thể

Tầng	Thành phần	Trách nhiệm chính
Input	CameraReader	Thread capture riêng; giữ frame mới nhất; reconnect với backoff; điều tiết file video theo FPS nguồn.
Vision	Shared YOLO inference API + per-camera ByteTrack	YOLO inference API/worker dùng chung model OpenVINO; ByteTrack và analytics state tách biệt theo camera.
Analytics	SingleCameraPipeline + analytics_service	Điều tiết FPS, vẽ overlay, đếm line crossing, phát hiện intrusion, đo hiệu năng.
Storage	EventDatabase / SQLite	Lưu event/snapshot và tổng count_in/count_out theo camera.
Output	FFmpegRTSPStreamer	Đẩy frame annotate tới RTSP server qua queue kích thước một và xử lý không đồng bộ.

Tầng	Thành phần	Trách nhiệm chính
Interface	FastAPI + PySide6	REST/MJPEG/API controls và desktop grid, counters, event table.
Operations	LoggingService	Log console và file rotating `logs/app.log`, ghi lỗi chưa xử lý ở thread.

3.1 Sơ đồ luồng

Thứ tự	Luồng
1	Nguồn camera/file → CameraReader (thread riêng, latest-frame buffer).
2	Frame mới nhất → shared YOLO inference API → detection → ByteTrack riêng của camera → line/zone analytics, event và SQLite.
3	Detection → line counter và zone detector → sự kiện SQLite/snapshot.
4	Annotated frame → FFmpeg publisher → RTSP endpoint; UI/MJPEG reads the RTSP output, as does VLC.
5	FastAPI cung cấp trạng thái, event, counts, start/stop, live MJPEG và snapshot.

3.2 Cách ly theo camera

Mỗi camera có pipeline, ByteTrack tracker và analytics state riêng để tránh trộn track ID. YOLO model/inference worker được chia sẻ qua API, giảm tải bộ nhớ so với một model mỗi camera; inference worker hiện xử lý tuần tự và là nút thắt throughput. SQLite dùng chung và được bảo vệ bằng lock trong tiến trình.

4. Công nghệ và cấu hình hiện tại

Thành phần	Thư viện / giá trị
Ngôn ngữ/runtime	Python; môi trường kiểm tra Windows; project yêu cầu Python 3.10+.
Computer vision	OpenCV, NumPy, Ultralytics YOLO.
Inference CPU	OpenVINO IR; yolov8s export; imgsz 480; FP16 export.
Tracking	Ultralytics với tracker mặc định của thư viện.
API	ByteTrack tracker riêng cho từng camera; YOLO model/inference worker được chia sẻ.
Desktop UI	PySide6 / Qt6.
Persistence	SQLite events + bảng camera_counts; snapshot JPEG.
Restream	FFmpeg + libx264 ultrafast/zerolatency + RTSP/TCP.
Config	YAML được parse và validate bởi Pydantic.

4.1 Cấu hình model/processing

Tham số	Giá trị	Diễn giải
weights	`resources/models/yolov8s_openvino_model`	OpenVINO model sẵn có được chọn làm target.
conf	0.7	Ngưỡng confidence; thấp hơn có thể tăng recall nhưng cũng tăng false positive.
iou	0.45	Ngưỡng IoU cho NMS; không nên nhầm với ngưỡng ghép track.
imgsz	480	Kích thước suy luận export; model OpenVINO được kiểm tra metadata trước khi reuse.
skip_frames	0	Infer ở mỗi frame pipeline xử lý; không có nghĩa xử lý đủ 30 FPS nguồn.
target_fps	12	Giới hạn tốc độ vòng pipeline/đầu ra; không phải throughput cam kết.
metrics_log_interval_seconds	10	Khoảng thời gian ghi nhận metrics theo camera.
device	cpu	OpenVINO CPU trong cấu hình hiện tại.

4.2 Export model

Ưu tiên sử dụng model với định dạng OpenVINO nhằm tối ưu performance trên CPU cũng như edge device. Hệ thống có xử lí convert định dạng Ultralytics (.pt) sang dạng OpenVINO để phù hợp với hệ thống.

5. Luồng xử lý video

5.1 Capture

- CameraReader chạy trong thread daemon và giữ một frame gần nhất thay vì queue dài; giảm độ trễ tích lũy nhưng chủ động bỏ frame cũ nếu inference chậm.
- Với file video, capture điều tiết theo FPS nguồn và lặp từ đầu khi tới cuối file; RTSP/webcam có cơ chế reconnect tăng dần.
- FPS input đo từ capture thread; output FPS đo từ vòng pipeline; inference FPS đo riêng để không nhầm ba đại lượng này.
- Shutdown chờ capture/pipeline có giới hạn timeout và ghi warning khi thread chưa dừng.

5.2 Pipeline và metrics

Pipeline lấy latest frame, quyết định có infer theo skip_frames, xử lý analytics ở lần infer, annotate, publish latest frame, gửi frame cho streamer và ghi metrics. Inference latency là trung bình theo từng cửa sổ log; processing latency là thời gian xử lý vòng lặp gần nhất. Metrics được trả trong GET /cameras và ghi log định kỳ.

5.3 Restream

FFmpeg nhận raw BGR frame qua stdin; queue maxsize=1 giữ frame mới nhất, tránh làm nghẽn loop vision. FPS pipeline được truyền làm input framerate; FFmpeg nhận wall-clock timestamp và chế độ passthrough. MediaMTX phải được chạy với mediamtx.yml có path publisher. Smoke test riêng xác nhận publish/read H.264 320×240; log chạy bốn camera có Broken pipe khi server/path không sẵn sàng, nên chưa xác nhận hoạt động bền vững đa camera.

6. Phát hiện, tracking và analytics

6.1 Detector và tracking

Model hiện là YOLOv8s pretrained COCO, dùng OpenVINO IR trên CPU và được chia sẻ qua inference worker. ByteTrack chạy riêng cho từng camera trên detections trả về từ model. Model chưa được training trên dữ liệu thật của camera; chất lượng detect/count chưa thể kết luận nếu thiếu ground truth.

6.2 Line crossing

- Điểm tham chiếu đối tượng là bottom-center của bounding box.
- Counter lưu vị trí về phía của đường cho từng track; chỉ đếm khi track đổi phía và quỹ đạo giao đoạn vạch.
- Direction in/out dựa vào hướng có dấu của đoạn vạch đã cấu hình. Ý nghĩa vào/ra vật lý cần thống nhất khi hiệu chuẩn từng camera.

6.3 Vùng hạn chế

Zone detector kiểm tra bottom-center với polygon và chỉ tạo event ở lần chuyển từ ngoài vào trong. Hysteresis 5 pixel ổn định trạng thái sát biên; track mất tạm thời không kích hoạt lại event ngay. Khi vật thể thực sự ra khỏi polygon rồi vào lại, sự kiện mới được phát. Polygon hiện được bật theo yêu cầu thử nghiệm nhưng có thể bao gồm một phần xe đang đỗ/chuyển động; đây là ROI sơ bộ, không phải semantic classification của “khu vực cấm”.

6.4 Cấu hình ROI tạm thời

Camera	Line (pixel nguồn)	Polygon vùng hạn chế (pixel nguồn)
cam_1	(500,760) → (1770,760)	[(560,650),(1740,650),(1870,860),(470,860)]
cam_2	(520,740) → (1780,740)	[(600,640),(1770,640),(1880,850),(490,850)]
cam_3	(500,780) → (1790,780)	[(570,675),(1780,675),(1900,875),(450,875)]
cam_4	(510,750) → (1760,750)	[(590,655),(1730,655),(1850,855),(470,855)]

7. Lưu trữ, API và giao diện giám sát

7.1 SQLite và snapshot

Bảng `events` lưu camera_id, track_id, event_type, details, đường dẫn snapshot và thời gian. Bảng `camera_counts` lưu tổng count_in/count_out theo camera. Khi mở DB, dữ liệu line crossing có trong `events` được backfill vào bảng totals; thao tác ghi line-crossing event và cập nhật tổng được thực hiện cùng transaction. Snapshot được ghi vào thư mục cấu hình. Tổng count tồn tại sau restart; trạng thái track/tracker không tồn tại qua restart.

7.2 REST API

Endpoint	Chức năng
GET /health	Health của API và inference worker; không đồng nghĩa toàn bộ camera, RTSP publish và database đều khỏe.
GET /cameras	Trạng thái camera, FPS, latency, reconnect count và tổng đếm runtime.
GET /counts	Tổng count in/out bền vững theo camera.
POST /cameras/{id}/start|stop	Bật/tắt pipeline camera.
GET /events	Truy vấn event, lọc camera/time và giới hạn số bản ghi.
GET /cameras/{id}/live	Luồng annotate MJPEG cho trình duyệt.
GET /snapshots/{filename}	Phục vụ snapshot từ thư mục snapshot.
GET /docs	Swagger/OpenAPI UI.
GET /inference	Trạng thái shared inference API/worker, queue và metrics inference.

7.3 Desktop UI

PySide6 UI hiển thị grid camera, trạng thái camera, bảng tổng đếm, sự kiện gần nhất và overlay. Live view tiêu thụ luồng annotate qua RTSP; bảng sự kiện đọc DB. Chế độ `python main.py --ui` chạy API nền cùng desktop app.

8. Đánh giá và kết quả kiểm thử

8.1 Functional tests

Nhóm kiểm thử	Phạm vi	Kết quả
Analytics	Analytics: line crossing, hysteresis/missing track, zone intrusion; ByteTrack state tách theo camera.	Pass.
Storage	Tổng count lưu bền vững, transaction/event, migration từ event cũ, kết nối SQLite đóng đúng.	Pass.
Detector	Shared OpenVINO model/API, reuse export, tự export và xử lý queue/timeout/error.	Pass.
Config	ROI camera-specific và tham số config.	Pass.
Streamer	Tham số framerate/wall-clock/fps passthrough được truyền vào lệnh FFmpeg.	Pass.
Tổng số test	Pytest trong thư mục tests.	32 passed.
Static sanity	Compile Python, parse config, validate MediaMTX config, RTSP publish/read smoke test và git diff --check.	Pass; RTSP smoke test riêng, không phải soak test bốn camera.

8.2 Inference smoke test

Chỉ số	Quan sát	Diễn giải
OpenVINO initialization	OpenVINO IR được load bởi shared inference service trên CPU; model runtime khởi tạo thành công.	Xác nhận đường OpenVINO, không chứng minh throughput đủ cho realtime.
Cold-start first inference	Phép đo pipeline một camera trước đây ghi nhận lần đầu khoảng 12.8 giây.	Có overhead khởi tạo/warm-up; đây là phép đo cũ và không dùng làm benchmark hiện hành.
Inference sau warm-up	Phép đo pipeline một camera trước đây	Không đáp ứng realtime; phép đo ngắn, không thể suy

Chỉ số	Quan sát	Diễn giải
	khoảng 1.28 giây/frame (~0.78 FPS).	rộng tuyến tính sang bốn camera.
Four-camera inference	Log đa camera ngày 24/10/2026: xấp xỉ 0.04 inference FPS/camera; latency trung bình theo cửa sổ 22.5–25.6 giây/frame.	CPU/inference là nút thắt rõ rệt; tracking/counting bị cập nhật quá chậm để coi là realtime.
Output	Output log gần nhất khoảng 3.2–7.9 FPS/camera.	Video output FPS không phải detection FPS; pipeline có thể phát frame nhanh hơn tốc độ infer.
RTSP integration	MediaMTX 1.21.2 config validate thành công; FFmpeg streamer của ứng dụng publish H.264 và RTSP reader nhận frame 320×240 trong smoke test riêng. Server default thiếu path publisher trả 400; log chạy ứng dụng sau đó có Broken pipe.	RTSP publish/read đã được xác minh với cấu hình đúng, nhưng chưa có soak test bốn camera. Luôn chạy MediaMTX bằng mediamtx.yml và kiểm tra publisher sau khi khởi động.

8.3 Độ chính xác nghiệp vụ chưa được đo

Chưa có dữ liệu ground truth gắn nhãn để tính precision/recall, IDF1 hoặc sai số đếm. Cần xây dựng tập kiểm thử đại diện cho từng camera trước khi đánh giá độ chính xác.

9. Hạn chế, rủi ro và khuyến nghị

Ưu tiên	Hạn chế / rủi ro	Khuyến nghị
P0	Bốn camera trên CPU chỉ đạt khoảng 0.04 inference FPS/camera, latency 22.5–25.6 giây theo log gần nhất.	Đo lại trên phần cứng xác định; benchmark 1/2/4 camera; thử kích thước/model/quantization trên cùng tập gắn nhãn và chỉ chấp nhận tối ưu khi sai số đếm được đánh giá.
P0	ROI hiện là sơ bộ; polygon vùng có thể bao xe đang đỗ/chạy, line IN/OUT phụ thuộc hướng.	Hiệu chuẩn riêng từng camera tại hiện trường; thống nhất quy ước hướng; lưu ảnh ROI đã duyệt và test count theo ground truth.
P1	Model COCO pretrained chưa tối ưu riêng bối cảnh, góc nhìn và kích thước xe/người.	Ghi lại missed detections/false positives và cân nhắc fine-tune với dữ liệu camera có nhãn.
P1	API bind 0.0.0.0; endpoint điều khiển và snapshot không có auth; CORS rộng.	Chỉ dùng mạng tin cậy hoặc thêm auth, giới hạn CORS/firewall, TLS/reverse proxy trước khi phơi ra LAN/Internet.
P1	Health endpoint không phản ánh đầy đủ trạng thái camera, RTSP publisher, tracker và storage.	Trả trạng thái degraded/down dựa trên camera heartbeat, model init, storage và streamer health.
P1	Tổng count là tổng tích lũy; chưa có reset/day shift/aggregation theo khoảng thời gian.	Xác định chính sách vận hành (reset theo ca/ngày hay tổng lịch sử), bổ sung key/time bucket nếu cần.
P2	Snapshot/event có thể tăng không giới hạn; API DB query có thể cần index.	Thiết kế retention, dung lượng snapshot, index camera_id/time và backup SQLite.
P2	Shared inference worker xử lý tuần tự; khi CPU quá tải, queue/timeout làm tăng thời gian chờ và có thể từ chối request.	Theo dõi queue depth/429/timeout; tối ưu model hoặc phần cứng trước khi cân nhắc batching/worker song song.
P2	RTSP publisher từng gặp HTTP 400/Broken pipe khi MediaMTX chạy thiếu cấu hình path.	Khởi chạy MediaMTX bằng cấu hình mediamtx.yml; thêm kiểm tra trạng thái publish và thử reconnect/client disconnect trong integration test.

9.1 Kế hoạch tối ưu theo giai đoạn

1. Giai đoạn 1 — đo baseline: chuẩn hóa hardware, OS, OpenVINO version, clip/test duration; đo warm-up riêng, infer latency p50/p95, CPU/RAM, output FPS cho 1/2/4 camera. Log hiện tại cho thấy khoảng 0.04 inference FPS/camera và latency 22.5–25.6 giây trong phép chạy bốn camera; cần benchmark lặp lại trên phần cứng ghi rõ.
2. Giai đoạn 2 — chọn model/runtime: so YOLOv8n/s, imgsz 320/416/480, FP16/INT8 bằng cùng dữ liệu gắn nhãn; không tối ưu FPS bằng cách bỏ qua kiểm tra count accuracy.
3. Giai đoạn 3 — hiệu chuẩn analytics: đặt line ở vùng đường xe chạy; polygon sát biên vật lý; xác minh điểm chân box và hướng count; đánh giá occlusion/ID switch.
4. Giai đoạn 4 — vận hành: bảo vệ API, health checks, retention/backup, giám sát queue/429/timeout và streamer; chạy soak test RTSP đa camera với MediaMTX cấu hình đúng.

10. Kết luận

Hệ thống có capture bất đồng bộ, shared YOLO/OpenVINO inference API, ByteTrack và analytics state riêng theo camera, SQLite, API, UI và RTSP streamer. 32 test tự động qua và RTSP publish/read đã smoke-test riêng với MediaMTX cấu hình đúng. Tuy nhiên throughput bốn camera hiện rất thấp (khoảng 0.04 inference FPS/camera;

latency 22.5–25.6 giây), log publisher còn Broken pipe, ROI cần hiệu chuẩn và counting accuracy chưa có ground truth. Chưa nên xem là hệ thống realtime hay production-ready cho đến khi các điểm này được khắc phục và xác minh.